

Homework2

- a. String vs StringBuilder vs StringBuffer
String is immutable and will store at constant String pool once created so use it when object doesn't need many modification.
StringBuilder is mutable and not thread safe so use it when many modifications at one thread.
StringBuffer is mutable and thread safe so use it when multiple thread modify the same string objects.
- b. Comparator vs Comparable, when to use which one
Both of them are interface used for sorting objects.
Comparator defines customer ordering outside the class using compare() method with multiple sorting logic.
Comparable defines natural ordering of objects inside the class using compareTo() method with one default sorting logic.
- c. Overriding vs overloading
Overriding: runtime polymorphism, subclass provides its own implementation of parent method.
Overloading: compile time polymorphism, occurs when same method with different signature(parameters)
- d. JRE vs JDK vs JVM
JDK -> JVM -> JRE
JDK is Java development kit for developer to develop Java application containing JRE and JVM.
JVM is Java virtual machine for running Java bytecode(a minimum unit).
JRE is Java runtime environment is used to run Java application includes JVM and runtime libraries.
- e. Java 8 basic data types
It includes Primitive types and reference types.
Primitive type(store simple values efficiently and faster than reference data type)
Integer Types: byte, short, int, long.
Floating point type: float, double.
Character type: char.
Boolean type: boolean.
- f. Primitive type, reference type
Primitive type specifies the type of a variable includes byte, short, int, long, float, double, boolean, char.

Reference type store object references.(all the other data types than Primitive type.)

String is a class reference type not primitive type.

Wrapper class includes Integer, Character, Double, Boolean. It used for Collections, Generics, and Utility methods.

- g. How does JVM work

JVM executes Java bytecode and converts it into machine code that the operating system can understand.

- h. JVM memory data model

Java Source Code(.java) - Compiler(.class) [inside JDK] - Bytecode - Class Loader - Bytecode Verifier - Runtime Memory Areas - Execution Engine - Machine Code

JVM implements Memory Area(RAM) -> Virtual machine: Method area, MV stack, Native Method stack, Heap, Program counter Register

- Method area stores source code, static (field, class, method)
- Heap stores real objects (ex: Constant String pool) without static
- VM stack stores reference point to object
- Program counter Register
- Native Method stack

- i. How does GC work

Garbage Collection automatically removes unused objects from memory to prevent memory leaks and free heap source.

- j. young/old/perm generation

Generational garbage collection: Java heap is divided into generations because most objects die young.

Young generation: stores newly created objects.

Old generation stores long-lived objects.

- k. difference types of GC

Common **GC types**:

- Serial GC (single-threaded, simple)
- Parallel GC (high throughput)
- CMS and G1 GC (lower pause time for enterprise applications)
- ZGC/Shenandoah (very low latency for large-scale systems)
- G1 GC is the default collector in modern Java because it balances performance and pause time efficiently.

Homework3

- a. Java modifier scope: public, private, protected, default scope
Access level
Public: everyone can access it.
Private: accessible only inside the class. Used for encapsulation or information hiding. Generally put all fields in private for information security.
Protected: accessible inside the same package or the same class. Subclasses even in different package can access it. Used when inheritance is involved.
Default scope: accessible in the same class and package, it's not accessible for different package.
- b. What is static scope
Static means the member belongs to class instead of objects. Static variables are shared among all instances, static method can be accessed without object creation.
Static block loaded in JVM only executed once for initialization.
Outer class cannot be static class.
- c. how does classloader work
ClassLoader is a part of JRE that loads .class bytecode file into JVM memory during runtime. It follows three phases: loading, linking, and initialization. There are three types of ClassLoader - Bootstrap ClassLoader, Extension ClassLoader, and Application ClassLoader.
- d. Describe the difference between unchecked and checked exceptions in Java.
Checked exception - compile time checking (before runtime) use throws and Exception clause to handle. Ex: IOExceptions
Unchecked exception - runtime exception.
Ex: Null Pointer Exception, Web applications - wrong user input, wrong format. Use to prevent user behavior to break backend server.
Database failure: customized exception
- e. What is the difference between finally, final, and finalize in Java?
Final is a keyword on class(can't be inherited), method, and variable(need initialize, can't re-reference) scope use to prevent modification.
Finally is a block of code used with try catch that always executes for clean up.
Finalize is called by garbage collector use to clean up code before object destruction.
- f. Define try-with resource. How can you say that it differs from an ordinary try?
Try-with resources introduced in Java 7 that automatically closing resources after use.
Ordinary try-catch where resources must be manually closed in a finally block, try-with resources handles cleanup automatically, reducing boilerplate code and preventing resource leaks.
- g. Define Runtime Exception. Describe it with the help of an example.
Exception contains two types: runtime and compile exceptions.

Runtime Exception occurs during program execution. It is also called an unchecked exception because the compiler does not force developers to handle it using try-catch or throws. It happens when invalid input, programming logic errors, invalid operations. Ex: NullPointerException, ArithmeticException.

Throwable: contains exception and errors(virtual machine error ex: electricity out of batteries, can do with program logic)

- h. What is the difference between NoClassDefFoundError and ClassNotFoundException in Java

ClassNotFoundException is a checked exception, occurs when JVM tries to load a class dynamically but cannot find it. Ex: missing .jar file or incorrect class path.

NoClassDefFoundError is an error, occurs when class was present during compilation but JVM but missing at runtime.

- i. Why should we clean up activities such as I/O resources in the finally block?
We need **in the finally block** cause finally block will always executes whether an exception occurs or not. This ensures resources such as files, sockets, database connections, and streams are properly closed to prevent resource leaks and memory issues.
- j. Describe OutOfMemoryError in exception handling.
OutOfMemoryError occurs when the memory available in the method area is not sufficient for the program startup, the JVM throws an error.

It is an Error indicating a serious JVM memory issue, commonly caused by memory leaks, excessive object creation, or insufficient JVM memory configuration.

To solve OutOfMemoryError:

- Identify which memory area is exhausted, such as heap, metaspace, or native thread memory.
- Analyze heap dumps or logs to find memory leaks, reduce unnecessary object creation, close resources properly, and tune JVM memory using options like -Xms and -Xmx.
- The best solution is not only increasing memory but also fixing the root cause.

- k. What is Generics in Java? What are the advantages of using Generics?
Generics in Java allow classes, interfaces, and methods to operate on parameterized types, providing compile-time type safety. They help eliminate explicit type casting, improve code reusability, and reduce runtime errors such as ClassCastException.

- l. How does Generics work in Java ? What is type erasure ?

Generics in Java work through type checking at compile time and type erasure at runtime. During compilation, the compiler verifies type safety and automatically inserts necessary casts.

After compilation, generic type information is removed and replaced with raw types or bounds such as Object or Number. This process is called **type erasure** and is used to maintain backward compatibility with older Java versions.

- m. What is the difference between List<? extends T> and List<? super T>?

List<? extends T> represents an upper bounded wildcard and is used when the list produces data. It allows reading elements as type T safely but does not allow adding elements.

List<? super T> represents a lower bounded wildcard and is used when the list consumes data. It allows adding objects of type T safely, but reading is only guaranteed as Object.

This follows the PECS principle: Producer Extends, Consumer Super.

- n. what is Optional class (write a demo code to use ofNullable, orElse, orElseThrow method)

Optional is a container class introduced in Java 8 to handle nullable values safely and reduce NullPointerException. ofNullable() creates an Optional that may contain null, orElse() returns a default value if empty, and orElseThrow() throws an exception when the value is absent.

- o. what is OOP

OOP is a programming paradigm that organizes code using objects and classes. Four main principles are Encapsulation, Abstraction, Inheritance, and Polymorphism.

Encapsulation hides internal information for sensitive data protection by using private final fields, no setters to restrict direct access.

Abstraction used abstract class or interface to hide implementation details and exposes only essential behaviors.

Inheritance using extends keywords for subclass to inherit fields and methods from a superclass promoting code reusability.

Polymorphism allows the same method name to behave differently in different situations. Overriding is Runtime polymorphism which is same method with different signature and overloading is a compile time polymorphism. Subclass has its own implementation of superclass method.

Coding Exercise 1

1. given a random character array, find the char with third highest frequency

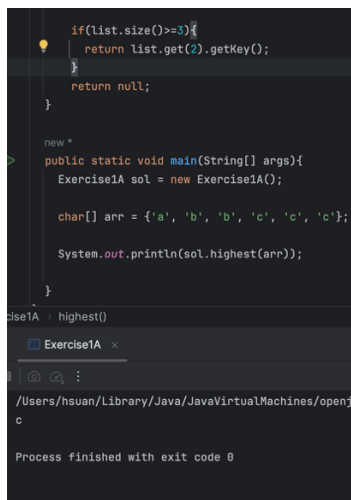
input: [a, b, b, c, c, c], output: [a]

```
public Character highest(char[] arr){  
    Map<Character, Integer> map = new HashMap<>();  
    PriorityQueue<Integer> pq = new PriorityQueue<>();
```

```
    for(char c : arr){  
        map.put(c, map.getOrDefault(c,0)+1);  
    }
```

```
    List<Map.Entry<Character,Integer>> list = new ArrayList<>(map.entrySet());
```

```
    if(list.size()>=3){  
        return list.get(2).getKey();  
    }  
    return null;  
}
```



```
    if(list.size()>=3){  
        return list.get(2).getKey();  
    }  
    return null;  
}  
  
new *  
> public static void main(String[] args){  
    Exercise1A sol = new Exercise1A();  
  
    char[] arr = {'a', 'b', 'b', 'c', 'c', 'c'};  
  
    System.out.println(sol.highest(arr));  
}  
  
Exercise1A : highest()  
Exercise1A x  
/Users/hsuan/Library/Java/JavaVirtualMachines/openj  
c  
Process finished with exit code 0
```

2. reverse a string
input: "abc", output: "cba"

```
9      Stack<Character> stack = new Stack<>();
10
11      char[] arr = str.toCharArray();
12      for(char c : arr){
13          stack.push(c);
14      }
15      while(!stack.isEmpty()) {
16          sb.append(stack.pop());
17      }
18      return sb.toString();
19  }
20  new *
21  public static void main(String[] args){
22      Exercise1B sol = new Exercise1B();
23      String str = "abc";
24      System.out.println(sol.reverse(str));
25  }
```

Exercise1B > reverse()

run Exercise1B x

/Users/hsuan/Library/Java/JavaVirtualMachines/op
cba

Process finished with exit code 0

3. given an integer array and target, return all the pairs sum to the target, each element can only be used once
input: [1, 2, 3, 4] target = 5, return [[1, 4],[2, 3]]

```
1 usage new *
2 public List<int[]> pair (int[] arr, int target){
3     List<int[]> ls = new ArrayList<>();
4     Map<Integer, Integer> map = new HashMap<>();
5
6     for(int num : arr){
7         map.put(num, map.getOrDefault(num, 0)+1);
8     }
9
10    for(int num : arr){
11
12    }
13 }
```

Exercise1C > pair()

Exercise1C x

/Users/hsuan/Library/Java/JavaVirtualMachines/openjdk-22.0.1

[1, 4]
[2, 3]

Homework4

- a. what is functional interface
Functional interface only has one abstract method and multiple default and static methods.
- b. what is default method
A **default method** is method inside an interface that already has implementation using default keyword. It allows developers to add new methods to interfaces without breaking existing classes that implement the interface.
- c. what is the difference between Predicate, Supplier, Consumer, Function?
Predicate used to test a condition and returns a boolean value. It takes one input and commonly used for filtering or validation.
Supplier is used to generate or supply data without taking any input. It only returns a value using the get() method.
Consumer used to consume or process data and returns nothing. It commonly used for printing, updating, or modifying data.
Function takes one input and transforms it into another result. It returns a value after processing the input.
- d. write a piece of code to use the Predicate, Supplier, Consumer, Function interface
Predicate<Integer> isEven = num -> num % 2 == 0;
sout(isEven.test(4)); // true

Supplier<Double> randomValue = () -> Math.random();
sout(randomValue.get());

Consumer<String> printName = name -> sout(name);
printName.accept("Alice");

Function<String, Integer> getLength = str -> str.length();
sout(getLength.apply("java"));
- e. what is method reference
Method reference is a shorthand syntax for lambda expression.
Ex: ClassName::methodName
There are three types of method references:

- static method Ex: Math::abs
- instance method Ex: System.out::println
- constructor method. Ex: ArrayList::new

f. what is CompletableFuture

A **CompletableFuture** is used for asynchronous non-blocking programming and multithreading. It allows tasks to run in the background without blocking the main thread.

```
Public static void main(String[] args) {
CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> {
    Return "Hello Async";
});
System.out.println(future.join());
}
```

g. Default keyword vs Java default scope

Default keyword is a keyword used inside interface to define methods with implementation. It helps maintain backward compatibility when adding new methods to interfaces.

Default scope, also called package-private scope, means no access modifier is specified. The class, method, or variable can only be accessed within the same package.

h. Coding: create a list of students, Student Class has name, age, score three fields.

```
List<Student> list = new ArrayList<>();
public class Main{
class Student{
    private String name;
    private int age;
    private int score;

    public Student(String name, int age, int score){
        this.name = name;
        this.age = age;
        this.score = score;
    }

    public String getName(){
        return name;
    }

    public int getAge(){
```

```

        return age;
    }

    public int getScore(){
        return score;
    }

}

public static void main(String[] args){
    List<Student> ls = new ArrayList<>();
    ls.add(new Student("Jack", 20, 95));
    ls.add(new Student("Andrew", 25, 98));
    ls.add(new Student("Alex", 28, 92));

    for(Student s: ls){
        System.out.println(s.getName() + " " + s.getAge() + " " + s.getScore());
    }
}
}

```

- use stream api to find all the students' name starting with 'A'

```

List<String> names =
students.stream()
    .map(Student::getName)
    .filter(name -> name.startsWith('A'))
    .collect(Collectors.toList());

```

- use stream api to get the sum of all the students score

```

int total =
students.stream()
    .mapToInt(Student::getScore)
    .sum();

```

- use stream api to find all the students **whose** score >= 60

```

List<Student> scores =
students.stream()
    .filter(student -> student.getScore() >= 60)
    .collect(Collectors.toList());

```

- use stream api to retrieve all students name

```
List<String> names =
    students.stream()
        .map(Student::getName)
        .collect(Collectors.toList());
```

- use stream api to count the frequency of each age

```
Map<Integer, Long> frequency =
    students.stream()
        .collect(Collectors.groupingBy(
Student::getAge,
Collectors.counting()
));
```

- use steam api to count the number of boys girls (groupby, collector.toMap())

```
Map<String, Long> number =
    people.stream()
        .collect(Collectors.groupingBy(
Student::getGender,
Collectors.counting()
))
    .entrySet()
    .stream()
    .collect(Collectors.toMap(
        entry -> entry.getKey(),
        entry -> entry.getValue()
    ));
```

- i. intermediate operation vs terminal operation

Intermediate operation is a stream and it return another stream. Ex: filter(), map(), sorted()

Terminal operation is a stream returns a result. Ex: collect(), count(), forEach(),

- j. thread lifecycle, how does thread transfer from one state to another

Thread lifecycle includes NEW, RUNNABLE, BLOCKED/WAITING, TIME_WAITING, TREMINATED.

NEW: thread is created but not started.

RUNNABLE: after calling start(), thread is ready to run.

BLOCKED: waiting for acquire a lock.

WAITING: waiting indefinitely for another thread.

TIME_WAITING: waiting for a specific time using sleep() or wait(timeout).

TREMINATED: thread finishes execution.

- k. Coding: given a char array, use stream api to **count the frequency of each char**

```
public static void main(String[] args) {  
    char[] arr = {'a','b','c','d','a'};  
  
    Map<Character, Long> countCharacter =  
        new String(arr)  
            .chars()  
            .mapToObj(c -> (char) c)  
            .collect(Collectors.groupingBy(  
                Function.identity(),  
                Collectors.counting()  
            ));  
  
    System.out.print(countCharacter);  
}  
}
```

1. Stream API: `map()` vs `flatMap()`;
map() is a stream used to transform each element into another element.(one to one mapping)
Ex: `List<String> names =
students.stream()
.map(Student::getName)
.collect(Collectors.toList());`

→ Here: Student object → String name

flatMap() used to flatten the nested streams or collections into a single stream.(one to many mapping)